COSC 6385 - Homework 1: Out-Of-Order (O3) Simulation

Nicholas Anderson

1 Problem Statement

For this assignment we were given sample python code which simulates out of order exeuctions using the tomasulo algorithm. This algorithm includes reorder buffer (ROB), register alias table (RAT), a load-store-queue, reservation stations, functional units, and pipelining. All of these features come together to facilitate out-of-order execution. We were tasked with modifying this code to allow for multiple instructions to be executed during one clock cycle. Additionally I implemented allowance for commiting multiple instructions per clock cycle, as that is something that we talked about during the lecture in which we went over a practice problem closely related to this homework.

Additionally the input and output described in the assignment pdf is different from what is currently implemented. Specifically the existing input cannot take in register values, number of reservation stations (RS), cycles in EX, cycles in MEM, number of functional units (FUs), ROB entries, and issue width from the input text file. The output does not output the final register and non-zero memory values. Therefore I added this functionality into the existing code.

2 Expanding Issue Width

For this assignment we expand issue width to be able to issue and commit up to 4 instructions in one cycle. Expanding the amount of commands which can be issued in a cycle can be done directly through main by modifying the issue stage (lines 90-105 in main.py). This is done by adding a for loop of range issue_width, and issuing an instruction each iteration as long as the issuing is valid (i.e. there is an open RS for the instruction) and there are still instructions to be issued.

Expanding issue with for commits as well is done through the `commit` function in commit.py. Again this is done simply by adding a for loop of range issue_width, and committing an instruction each iteration as long is the head pointer of the ROB is pointing at a instruction that is ready to commit. Since instructions must commit in order, as soon as the instruction which the head pointer points to (implemented as `ROB[0]`) is not ready to commit, we break from the fop loop so no further operations are committed and the current operation is not removed from the ROB.